Design Rationale Requirement 2

Design 1

1. Each Animal class, like Bear, Wolf, Deer, Crocodile, and Spawner, like Cave, Swamp, is directly added to each map
2. To add Crocodile and Swamp, explicit code had to be inserted in each map's initialization sequence
3. Special spawning capabilities such as spawn deer needing to spawn an apple, had to be hardcoded inside each spawner
4. The YewBerryTree had no detection logic for nearby actors, and adding unique behavior required implementing the tick () logic again for the specific tree.

Pro	Cons
Simple to implement and readable	High connascence that adding a new animal or spawner requires touching multiple locations in code
	Repetitive spawning logic and drop behavior repeated for each animal type, which violates DRY
	The Spawner class had mixed all the logic, like creation, special behavior, and effect, which violates the Single Responsibility Principle
	Hard to extend as it needed to be modified multiple times in the existing code

Design 2

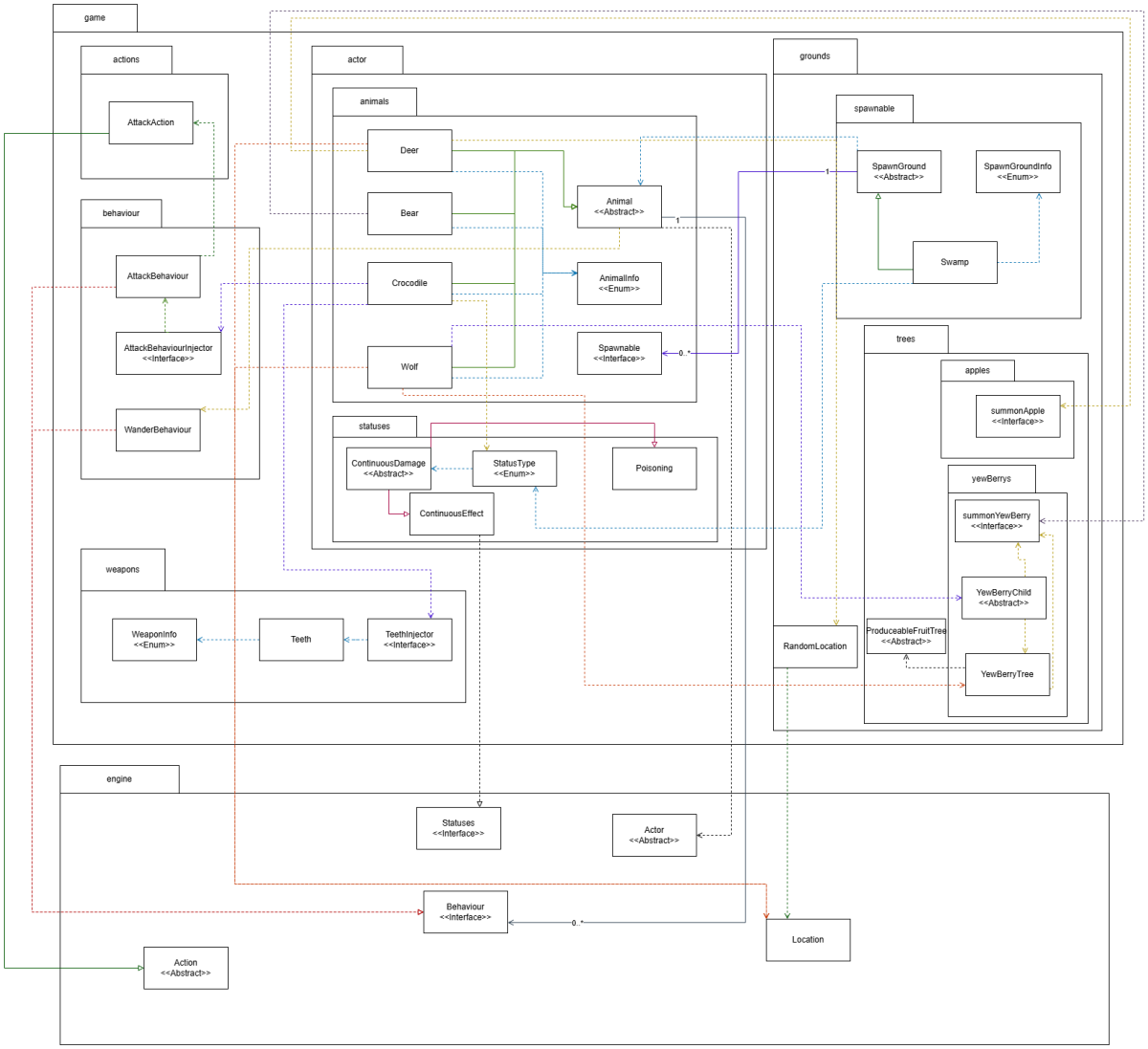
- Introduced spawner and animal structure using the existing Spawnable interface
- Add a Crocodile that extends the Animal parent class, which implements the Spawnable interface
- Add a Swamp class that extends SpawnGround as a new spawn ground
- Extend the spawning capabilities by adding the marker inside the ProduceableFruitTree and SpawnGround to implement the detect nearby logic
- Implement detectMode flag in ProduceableFruitTree used for trees spawned by Wolf to avoid the need to create a specific tree class and prevent duplication of code, like overwriting the tick()
- implement the injector interface, such as AttackBehaviourInjector and TeethInjector, which follow the Dependency Inversion Principle to eliminate direct dependency between concrete classes
- Teeth class defines crocodile's intrinsic weapon behaviour

Additional:

- WeaponInfo enum to remove the magic number inside the Teeth

Pro	Cons
Follow the SOLID principle: - SRP where each animal and spawner focuses on one responsibility. Swamp handle creation, Animal handle behavior, Injector handle instantiation - OCP where new animals and spawners can be added without modifying the existing code - LSP where the new injector behaves consistently as an abstraction provider - DIP where grounds depend on abstractions, not concrete animals	More complex than naïve design
Follow the OOP, which improves the maintainability	
Eliminate the connascence where the spawner manages the creation and linking internally	
Low coupling and high cohesion, where the spawner only manages creation and effect application, and animals manage their own actions	
Adhere to DRY and KISS, reducing the redundancy and improving readability	

Chosen Design:



Design 2,

As it provides clarity, maintainability, and scalability. It eliminates connascence by internalizing spawning logic in modular spawner and factory structures. It supports new animals, spawners, and effects with minimal changes. It follows the OOP, SOLID, DRY, and KISS principles. Animals and spawners can be added easily without breaking the existing code.

Existing Class and Responsibilities

Interface:

- AttackBehaviourInjector -> return new AttackBehaviour instances for animals
- TeethInjector -> return new instance of Teeth to avoid direct new Teeth() dependency

Abstract class:

- ProduceableFruitTree (extend from req1) -> Have a flag to summonFruit in two logic
- SpawnGround(extend) -> implement a flag to detect nearby actor

Concrete class:

- Teeth -> define crocodile's intrinsic weapon
- AttackBehaviour -> implement auto attack against the nearby actor
- Crocodile, Bear, Deer, Wolf -> animal with specific spawn capability when spawned in each spawner
- Swamp -> detect nearby actors and spawners, poisoned animal
- Poisoning -> apply continuous poison effect on actor

Class Interaction and Functionalities

- Swamps use detectActor() to determine an actor nearby and spawn a specific animal
- TeethInjector and AttackBehaviourInjector inject their intrinsic weapon and attack behavior when Crocodile spawns
- AttackBehaviour auto search surroundings to attack
- Poisoning status applies via post-attack logic and swamp spawning behavior
- Animals have spawnCapability for each animal to extend and perform specific actions

Deliver Functionalities

1. Crocodile engages the nearby actor using its injected intrinsic weapon
2. Teeth and AttackBehaviour apply attack resolution damage and trigger post-attack logic
3. Swamp detects nearby actor and has with 50% to spawn a poisoned animal
4. If an actor is nearby when the crocodile spawns, the actor is poisoned for three turns
5. When wolves spawn, ProduceableFruitTree will change detectMode to true to set the spawn YewBerry to detect the nearby actor instead of the turns of the round
6. Spawning logic for each animal is consistent across all spawner types

Conclusion

The second design, the injector interfaces, encapsulate the creation and post-attack logic, which eliminates the direct dependency. This internalization of creation order

removes the connascence of execution and reduces coupling between spawner and animal classes. It follows the OOP principle and achieves low coupling and high cohesion by separating the spawning logic, attack behavior, and weapon definition through abstraction and dependency injection. It minimizes repetitive code and supports future extension and prevents casting or direct dependencies between concrete classes. It aligned with SOLID, DRY, and KISS principles and fulfilled REQ2's requirement.